PokeService
- singleton: PokeService
- library: HashMap<PokeEnum, PokemonInfo>

+ getSingleton(): PokeService
+ getAvailablePokemons(): List<PokemonInfo>
+ getInfo(PokeEnum id): PokemonInfo
+ getStarters(): List<PokemonInfo>
- PokeService()
- register()
- registerInfo(PokeEnum id, PokemonInfo info)

Ottiene

1 — Pokemon registrati — 0..*

<<Enum>> PokeEnum
NONE
BULBASAUR
IVYSAUR
VENUSAUR
CHARMANDER
...
MEW = 151

<<Interface>> PokemonInterface
+ getName(): String
+ getId(): PokeEnum
+ getElement(): PokeElement
+ getLevelType(): LevelUpGroup
+ getCurrentEvolution(): int
+ getMaxEvolution(): int
+ getDropExp(): int
+ hasEvolution(): boolean
+ evolveAtLevel(): int

<<Enum>> PokeElement
NORMAL
FIRE
...
DRAGON = 14

<<Interface>> MoveInterface
+ getName(): String
+ getElement(): PokeElement
+ getType(): MoveType
+ getMaxUses(): int
+ getPower(): int
+ getAccuracy(): int
+ getRequiredLevel(): int
+ perform(Pokemon attacker, Pokemon defender): MoveResult

<<Enum>> MoveType
MISS
PHYSICAL
SPECIAL
STATUS

MoveResult
- damage: int
- moveResult: AttackEffectiveness
- elemResult: AttackEffectiveness
- critical: boolean
- effects: PokeStats
- tpye: MoveType

+ MoveResult(int, boolean, AttackEffectiveness, AttackEffectiveness)
+ MoveResult(PokeStats effects)
+ MoveResult()
+ getType(): MoveType
+ getEffects(): PokeStats
+ getDamage(): int
+ getMoveResult(): AttackEffectiveness
+ getElemResult(): AttackEffectiveness
+ isCritical(): boolean

2

<<Enum>> AttackEffectiveness
MISS
NOT_EFFECTIVE
NORMAL
EFFECTIVE;

+ getText(): String
+ getEffectiveness(): double

PokemonInfo
baseStats: PokeStats
+ getRequiredExp(int level): int

1 — Informazioni base — 1

Implementazioni

Squirtle | Squirtle | Squirtle
Wartortle | Wartortle | Wartortle
Blastoise | Blastoise | Blastoise

Pokemon
- moves: PokeMove[4]
- health: int
- iv: PokeStats
- ev: PokeStats
- effects: PokeStats
- level: int
- exp: int
- info: PokemonInfo

+ method(type): type

1 — Detiene Mosse — 0..4

3 — Statistiche base — Statistiche personali

0..6 — Detiene dei Pokemon

PokeMove
- move: BaseMove
- usesLeft: int
- slot: int

+ PokeMove(BaseMove)
+ getSlot(): int
+ setSlot(int)
+ getUsesLeft(): int
+ consumeUses()
+ resetUses()
+ canUse(): boolean

1 — Informazioni base della mossa — 1

0..1 — Mossa dell'azione — 1

BaseMove
- requiredLevel: int

+ BaseMove(int)
+ getRequiredLevel(): int
+ getEffects(): PokeStats
+ calculateDamage(Pokemon attacker, Pokemon defender): MoveResult

0..* — Mosse registrate — 1

Implementazioni

Growl | Tackle | Ember | TailWhip | VineWhip | WaterGun | Scratch | DragonBreath | DragonClaw

Use

MoveService
- singleton: MoveService
- library: HashMap<PokeEnum, ArrayList<BaseMove>>

+ getSingleton(): MoveService
+ getMoves(PokeEnum): List<BaseMove>
+ getMoveAtLevel(PokeEnum id, int level): BaseMove?
+ getSupportedMoves(Pokemon): List<BaseMove>
- MoveService()
- registerMoves()
- registerMove(PokeEnum id, BaseMove move)

Il pokemon che si vuole schierare — 1

<<Interface>> StatsInterface
+ getPS(): int
+ getAttack(): int
+ getDefense(): int
+ getSpecial(): int
+ getSpeed(): int

PokeStats
- attack: int
- defense: int
- special: int
- speed: int
- ps: int
- maxValue: int

+ PokeStats(int maxValue)
+ PokeStats()

Trainer
- pokemons: Pokemons[6]
- name: String
- currentPokemon: int

+ getTeamSize(): int
+ getPokemons(): Pokemon[]
+ getPokemon(int slot): Pokemon
+ setPokemon(int index, Pokemon pokemon)
+ addPokemon(Pokemon)
+ getActivePokemon(): Pokemon
+ setActivePokemon(int index)
+ setActivePokemon(Pokemon)
+ getAvailablePokemons(): List<Pokemon>
+ getAllPokemons(): List<Pokemon>
+ getFirstAvailablePokemon(): Pokemon
+ getTeamLevel(): int
+ getName(): String
+ setName(String)
+ getBestEvolution(): int

Actors — 0..2

BattleAction
- type: BattleActionType
- trainer: Trainer
- opponent: Trainer
- pokemon: Pokemon
- move: PokeMove

+ BattleAction(Trainer, Trainer, PokeMove)
+ BattleAction(Trainer, Pokemon pokemon)
+ BattleAction()
+ getType(): BattleActionType
+ getTrainer(): Trainer
+ getOpponent(): Trainer
+ getPokemon(): Pokemon
+ getMove(): PokeMove
+ performAction(): MoveResult?
- BattleAction(Trainer trainer, Trainer opponent)
- applyAttack(MoveResult attack)

2

L'azione che si sta eseguendo

<<Enum>> BattleActionType
SKIP
ATTACK
POKEMON_SWITCH

Azioni battaglia ordinate per priorita'

<<Enum>> Difficulty
EASY
MEDIUM
HARD
IMPOSSIBLE
HAHAHAHAH

return

GameState
- singleton: GameState
- player: Trainer
- score: int

+ getSingleton(): GameState
+ increaseScore()
+ getScore(): int
+ resetScore()
+ setPlayer(Trainer)
+ getPlayer(): Trainer
+ getCurrentDifficulty(): Difficulty

PlayerScore
- name: String
- score: int

+ PlayerScore(String, int)
+ getName(): String
+ getScore(): int

0..10 — Lista di punteggi registrati — 1

Leaderboard
- singleton: Leaderboard
- scores: ArrayList<PlayerScore>

+ getSingleton(): Leaderboard
+ getMinScore(): int
+ getScores(): List<PlayerScore>
+ addScore(String, int)
+ addScore(PlayerScore)
+ save()

Use

PokeFactory
- random: Random
- minLevel: int
- maxLevel: int
- maxPS: int
- maxAttack: int
- maxDefense: int
- maxSpecial: int
- maxSpeed: int
- id: PokeEnum

+ PokeFactory(PokeEnum)
+ PokeFactory()
+ setId(PokeEnum)
+ setMinLevel(int)
+ setMaxLevel(int)
+ generate(): Pokemon
- generateMoves(Pokemon)

Use

TrainerFactory
- pokeFactory: PokeFactory
- randomizer: PokeRandomizer
- random: Random
- generateTeam: boolean
- useStarters: boolean
- minTeamSize: int
- maxTeamSize: int
- NAMES: String[]

+ generate(): Trainer
- generateName(Trainer)
- generateTeam(Trainer)
- getRandomName(): String
- setMinTeamSize(int)
- setMaxTeamSize(int)
- setMinPokeLevel(int)
- setMaxPokeLevel(int)
- setMinEvolution(int)
- setMaxEvolution(int)
- useStarters(boolean)

1 — Generazione Team — 1

PokeRandomizer
- service: PokeService
- random: Random
- minEvolution: int
- maxEvolution: int

+ setMaxEvolution(int)
+ setMinEvolution(int)
+ getAvailablePokemons(): List<PokemonInfo>
+ getRandomInfo(int size): PokemonInfo[]
+ getRandomUniqueInfo(int size) PokemonInfo[]
+ getRandomStarters(size): List<PokemonInfo>

Selettore di Pokemon — 1

TurnHandler
- player1: Trainer
- player2: Trainer
- player1Action: BattleAction
- player2Action: BattleAction

+ setPlayer1Action(BattleAction)
+ setPlayer2Action(BattleAction)
getOrderedActions(): List<BattleAction>

1

BattleHandler
- state: GameState
- cpuHandler: CPUHandler

+ BattleHandler(GameState)
+ getCPU(): Trainer
+ generateNewOpponent()
+ startTurn(): Queue<BattleAction>
+ playerCatchPokemon(): Pokemon
- setupCPU()

<<Interface>> CPUInterface
+ handlePokemonChange(): Pokemon
+ handleTurn: BattleAction

CPUHandler
cpu: Trainer
opponent: Trainer
random: Random

+ CPUHandler(Trainer cpu, Trainer opponent)
makeAction(Pokemon): BattleAction
makeAction(PokeMove): BattleAction

EasyCPUHandler

Application

User

JPokeBattle
+ WIDTH: int
+ HEIGHT: int
- instance: JPokeBattle
- stage: Stage

+ getInstance(): JPokeBattle
+ setView(String)
+ getScene(): Scene
+ getStage(): Stage

Mostra

Controllers

StartController — Primo Input — MenuController

LeaderController — Indietro / Classifica — MenuController

FinishController — Fine — MenuController

SetupController — Nuova Partita — MenuController

SetupController — Indietro — MenuController

SetupController — Inizio — BattleController

MenuController — Indietro / Informazioni — InfoController

MenuController — Esci/Fuggi[score==0] — BattleController

BattleController
- game: GameState
- battleHandler: BattleHandler
- actions: Queue<BattleAction>
- lastAction: BattleAction
- freeSwitch: boolean
- player: Trainer
- catchingPokemon: Pokemon
- learningMove: PokeMove

+ runAway()
- startTurn(BattleAction playerAction)
- handleAction(BattleAction action)

Use — 1

SoundService
- singleton: SoundService
- effects: ArrayList<AudioClip>
- musics: ArrayList<AudioClip>

+ getSingleton(): SoundService
- loadAudio(String name, SoundType)
+ playPurchase()
+ playPress()
+ getTitleAudio(): AudioClip
+ getBattleAudio(): AudioClip
+ getVictoryAudio(): AudioClip

Use

GUI

ArrowTextButton
- button: Button
- arrow: ImageView

+ ArrowTextButton()
+ ArrowTextButton(String text, boolean enabled)
+ removeArrow()
+ addArrow()
+ getButton(): Button
+ setText(String)
+ showArrow(boolean)
+ resizeArrow(int width, int height)

ArrowButton
- arrow: Image
- imageView: ImageView

+ ArrowButton(int width, int height)
+ ArrowButton()
+ resize(int, int)

GridMenu
- options: MenuOption[]
- selected: MenuOption
- clickCallback: MenuCallback
- selectCallback: MenuCallback

+ GridMenu()
+ update(MenuOption[] options, int cols)
+ selectOption(MenuOption)
+ setArrowSize(int, int)
+ setClickCallback(MenuCallback)
+ setSelectCallback(MenuCallback)
- handleClick(MenuOption option)
- handleMouseHover(MenuOption option)

0..* — **MenuOption**
- text:String
- enabled: boolean
- value: int
button: ArrowTextButton

+ getText(): String
+ setText(String)
+ isEnabled(): boolean
+ setEnabled(boolean)
+ getValue(): int
+ setValue(int)
+ showArrow(boolean show)

HealthBar
- healthRect: Rectangle
- tweenTimeline: TimeLine

+ HealthBar()
- calculateHealthWidth(Pokemon)
- getColorFromHealth(Pokemon)
+ updateColor(Pokemon)
+ tweenHealth(Pokemon, PokeCallback)
- stopAnimation()
+ updateHealth(Pokemon)

HPokeBalls

+ HPokeBalls()
- getPokeBalls(): ImageView
+ update(Trainer)

LeaderboardGUI
- tableView: TableView<PlayerScore>

+ LeaderboardGUI()
+ update(List<PlayerScore>)

PokeCard
- pokemonView: PokemonView
- pokemonName: Label
- levelLabel: Label
- psLabel: Label
- arrow: ImageView
- healthBar: HealthBar

+ PokeCard()
+ toggleArrow(boolean)
+ update(Pokemon)

PokeInfoCard
- nameLabel: Label
- hpLabel: Label
- levelLabel: Label
- pokemon: Pokemon
- healthBar: HealthBar

+ PokeInfoCard()
+ update(Pokemon)
+ tweenHealth(PokeCallback onFinished)
+ updateHealth()
+ removeHpLabel()
+ getHealthBar(): HealthBar
- updateHealthLabel()

PokemonView
- id: PokeEnum
- renderBack: boolean

+ updateImage()
+ setPokemon(PokeEnum)
+ setRenderBack(boolean)

<<Interface>> ClickCallback
+ onClick(int, Pokemon)

Use

PokeTeamList
- currentCard: PokeCard
- callback: ClickCallback

+ PokeTeamList()
+ update(Trainer)
+ setCallback(ClickCallback)
- toggleCurrentCardArrow()